THE UNIVERSITY OF MELBOURNE

COMP30023 – Computer Systems

TCP's sliding window

Dr Lachlan Andrew

©2025 University of Melbourne

Recap

- TCP services
- TCP initialization
- TCP tear-down

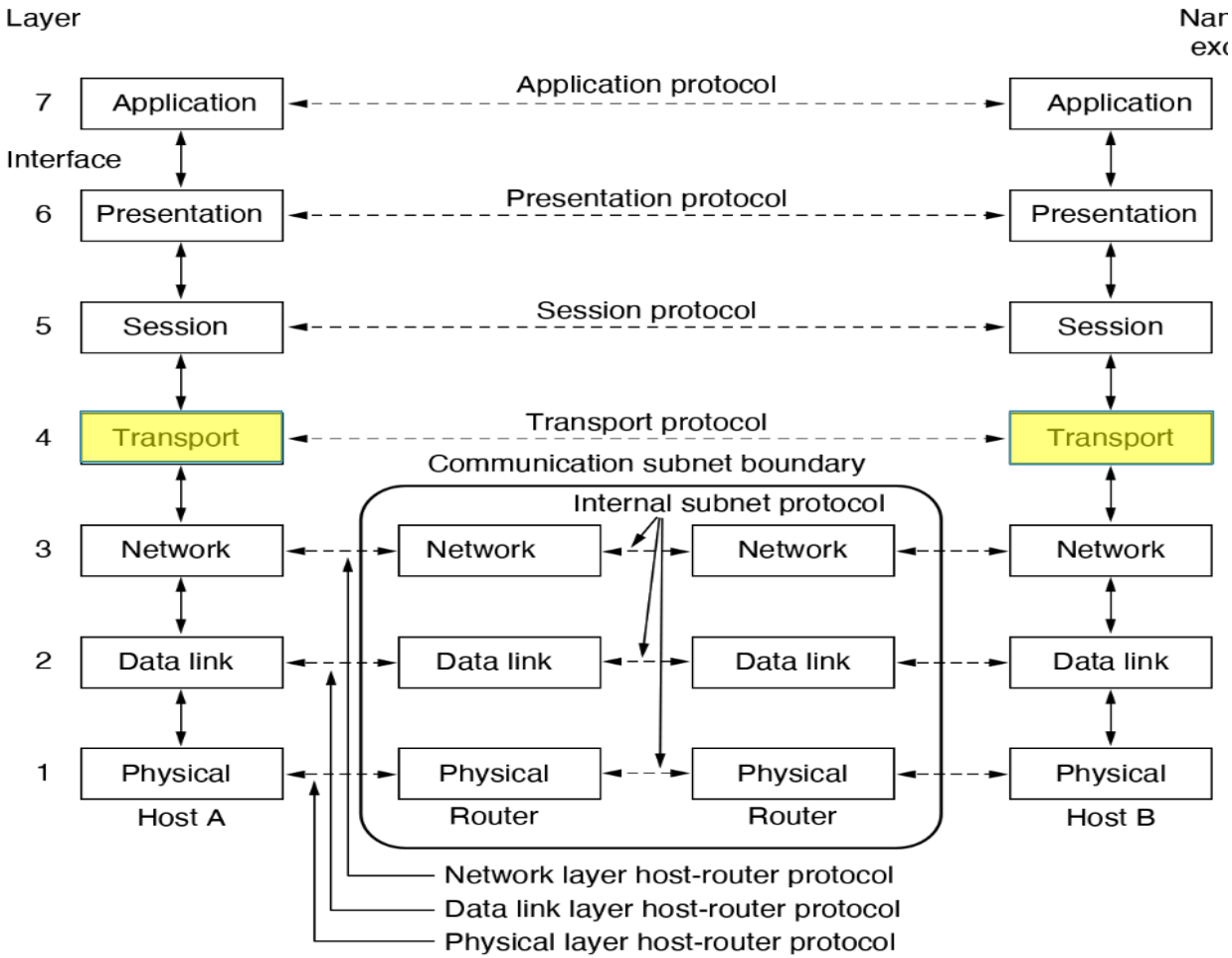
TCP: Sending and receiving data

```
    // #define required for VSCode
#define _POSIX_C_SOURCE 200112L
#include <netdb.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

int connfd = 0, re = 1, s, n;
struct addrinfo hints, *res, *rp;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_INET6;
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_ACTVE;
```

```
s = getaddrinfo("localhost",
                "5000", &hints, &res);
for (rp = res; rp != NULL;
            rp = rp->ai_next) {
    connfd = socket(rp->ai_family,
                    rp->ai_socktype,
                    rp->ai_protocol);
    if (connfd == -1) continue;
    if (connect(connfd,
                rp->ai_addr,
                rp->ai_addrlen) != -1
        break;
    close(connfd);
}
write (connfd, "Hello, network!\r\n");
close (connfd);
```

© 2025 University of Melbourne

Position in the stack: Transport layer



- Use data
- Tidy up e2e
- Get data e2e
- Tidy up p2p
- Get data p2p

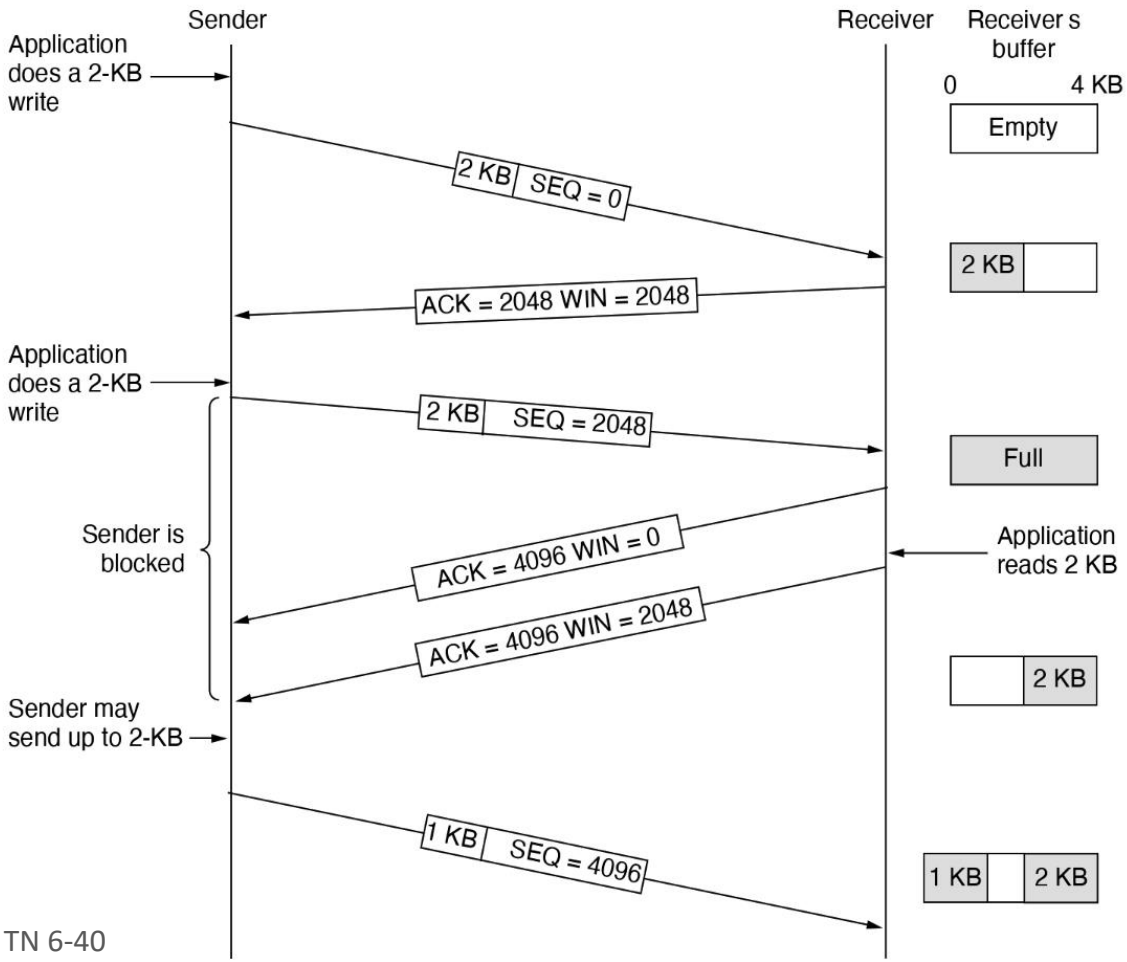
TCP Sliding Window

- Sliding window provides
  - Flow control (not sending too much to the receiver)
  - Reliable delivery
  - In-order delivery

- Sliding window is controlled by receiver
- Determines amount of data the receiver is able to accept
  - Sender and receiver maintain buffers to send and receive data independently of the application
  - No guarantee that data is immediately sent or read from the respective buffers

© 2025 University of Melbourne

TCP Sliding Window



TN 6-40

TCP Sliding Window

- When the window is 0 the sender should not send any data
  - Can send URGENT data
  - Can send ACK packets, with 0 bytes of data
  - Can send “zero window probe”: 0 byte segment that causes the receiver to re-announce the next expected byte and window size (window probe) this is designed to prevent deadlock
- Senders may delay sending data, e.g., instead of sending 1000B immediately, could wait for a further 500B to fill the a 1500B packet

© 2025 University of Melbourne

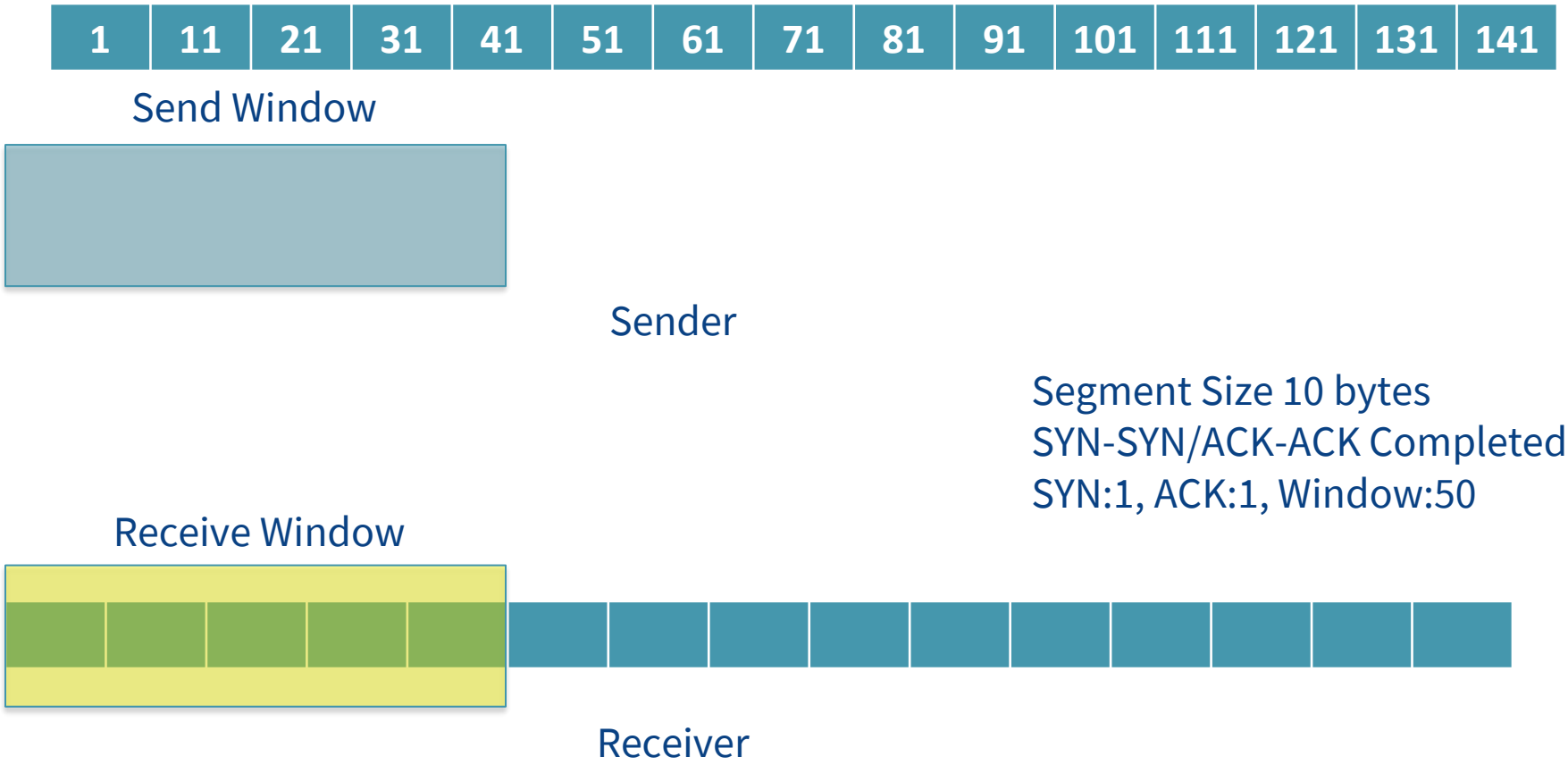
TCP Sliding Window

- Send window
  - What data the sender is able to send – unacknowledged segments and unsent data that will fit into the receive window
- Receive window
  - Amount of data the receiver is willing to receive – window size in ACK

- Other windows are maintained for congestion control

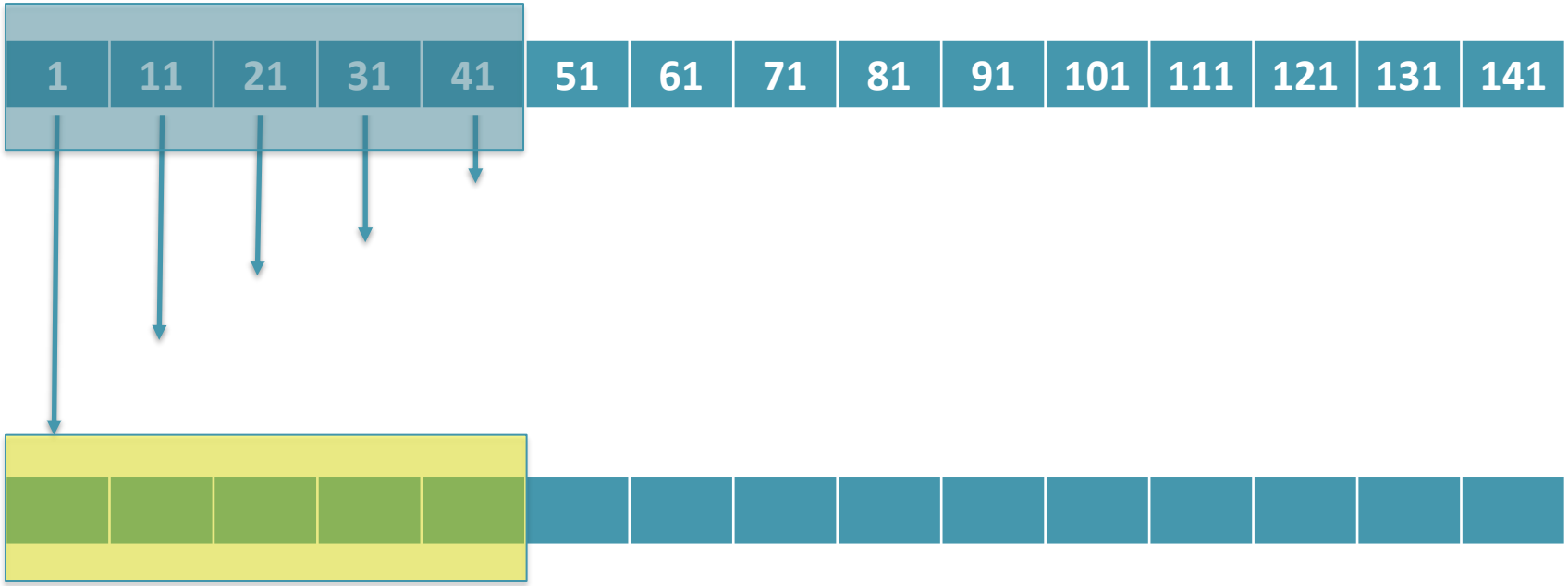
© 2025 University of Melbourne

TCP Sliding Window



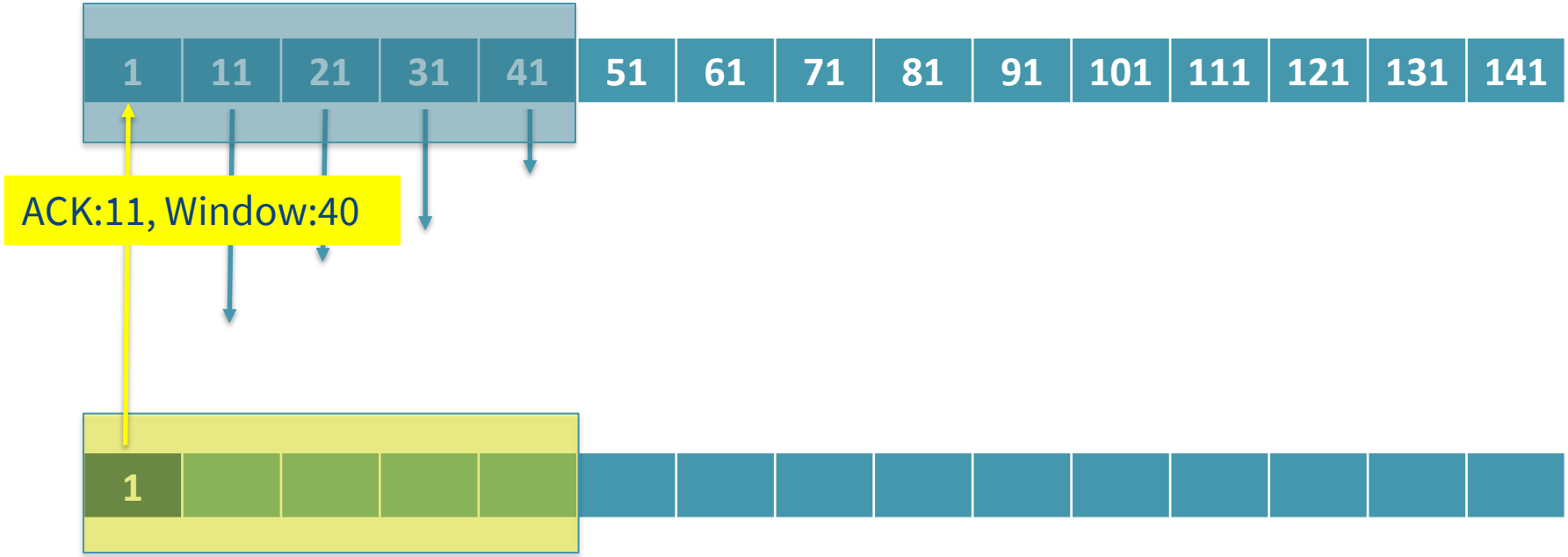
© 2025 University of Melbourne

TCP Sliding Window

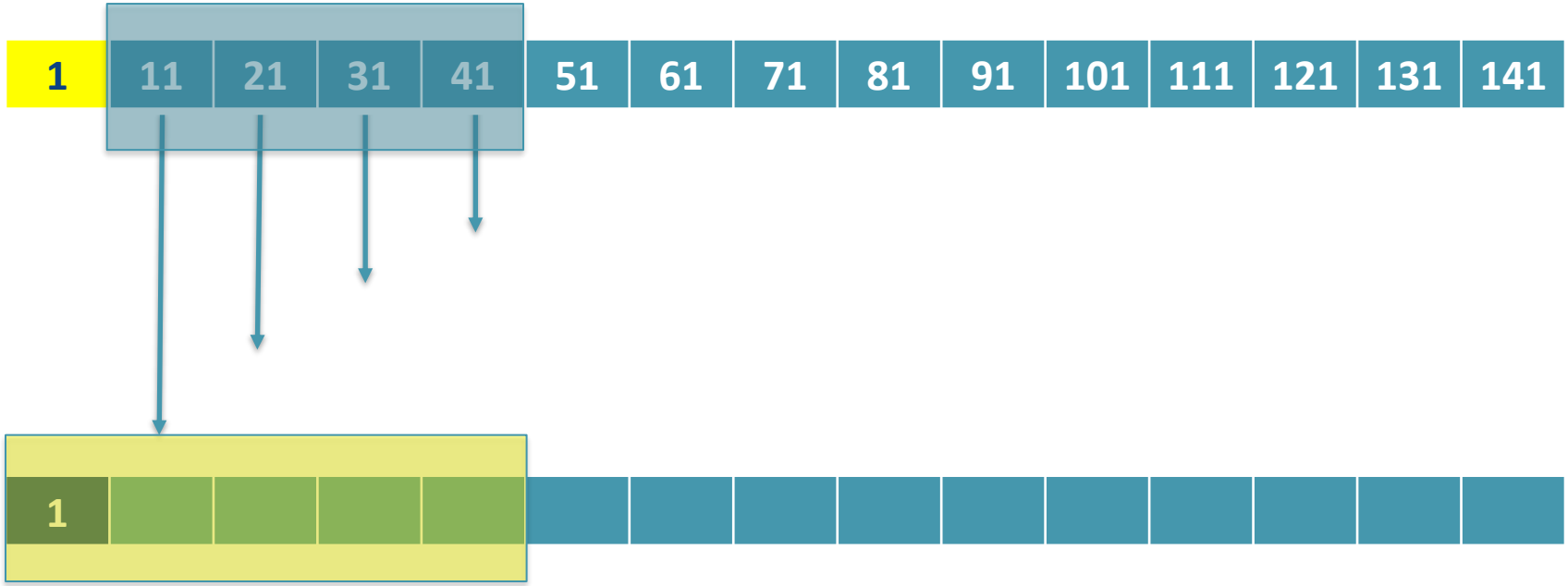


© 2025 University of Melbourne

TCP Sliding Window

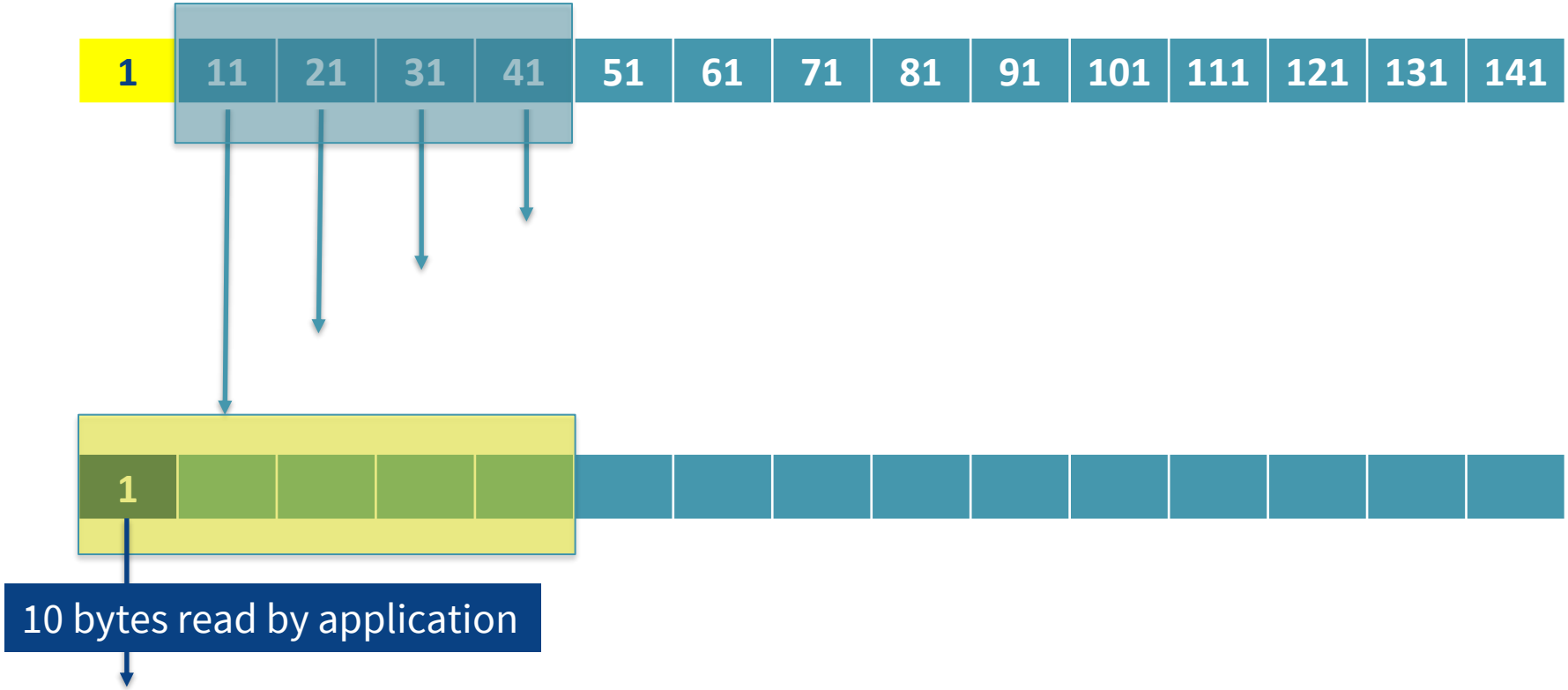


TCP Sliding Window



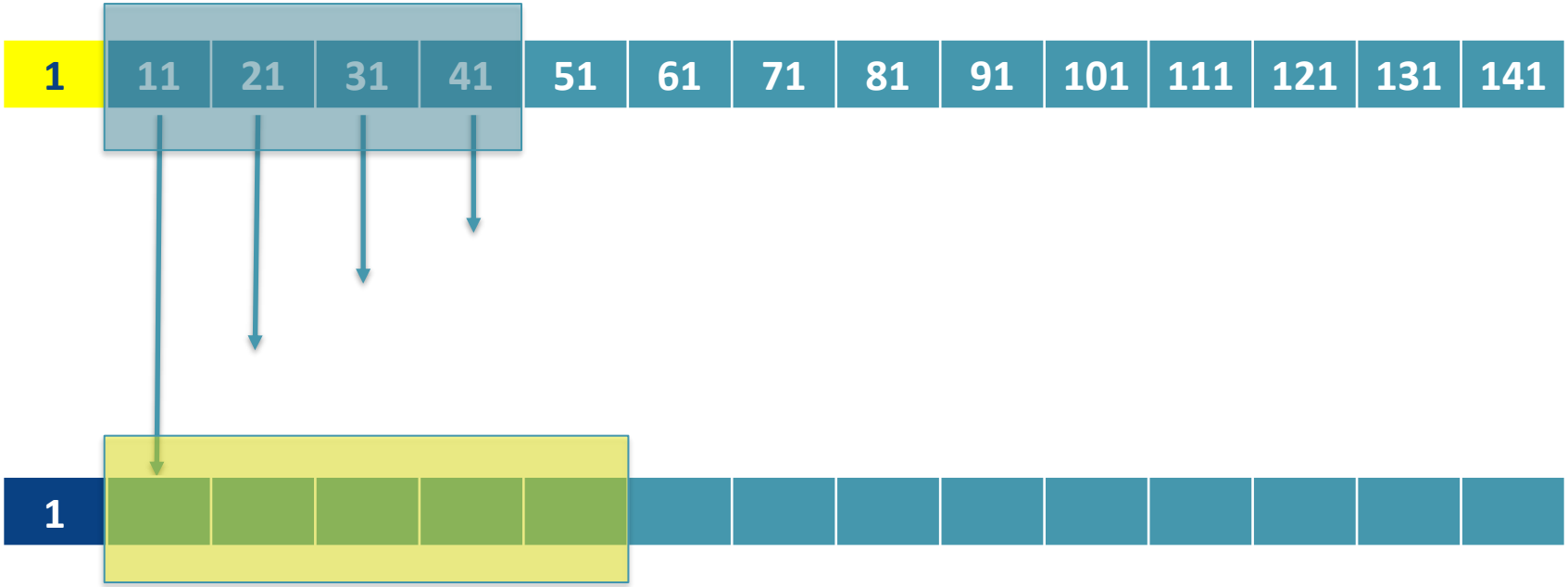
© 2025 University of Melbourne

TCP Sliding Window



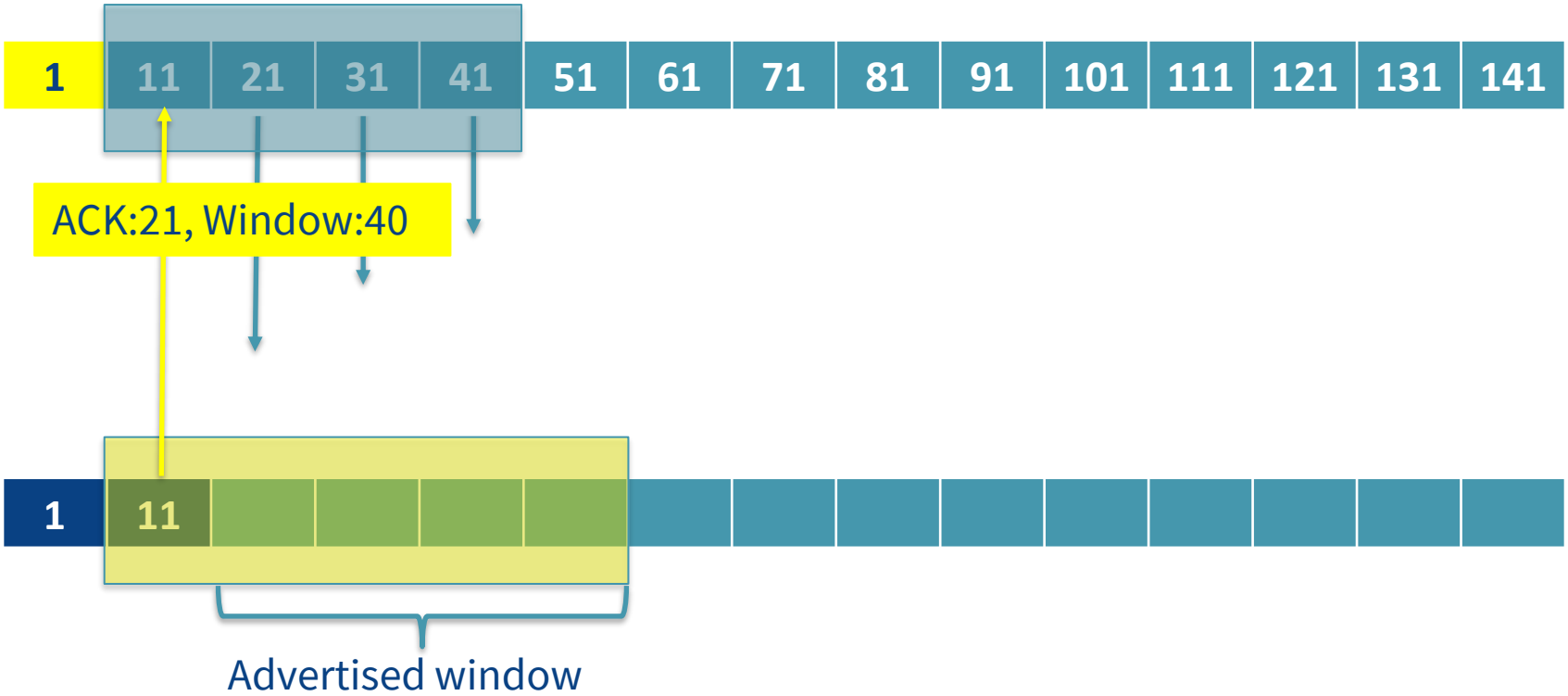
© 2025 University of Melbourne

TCP Sliding Window



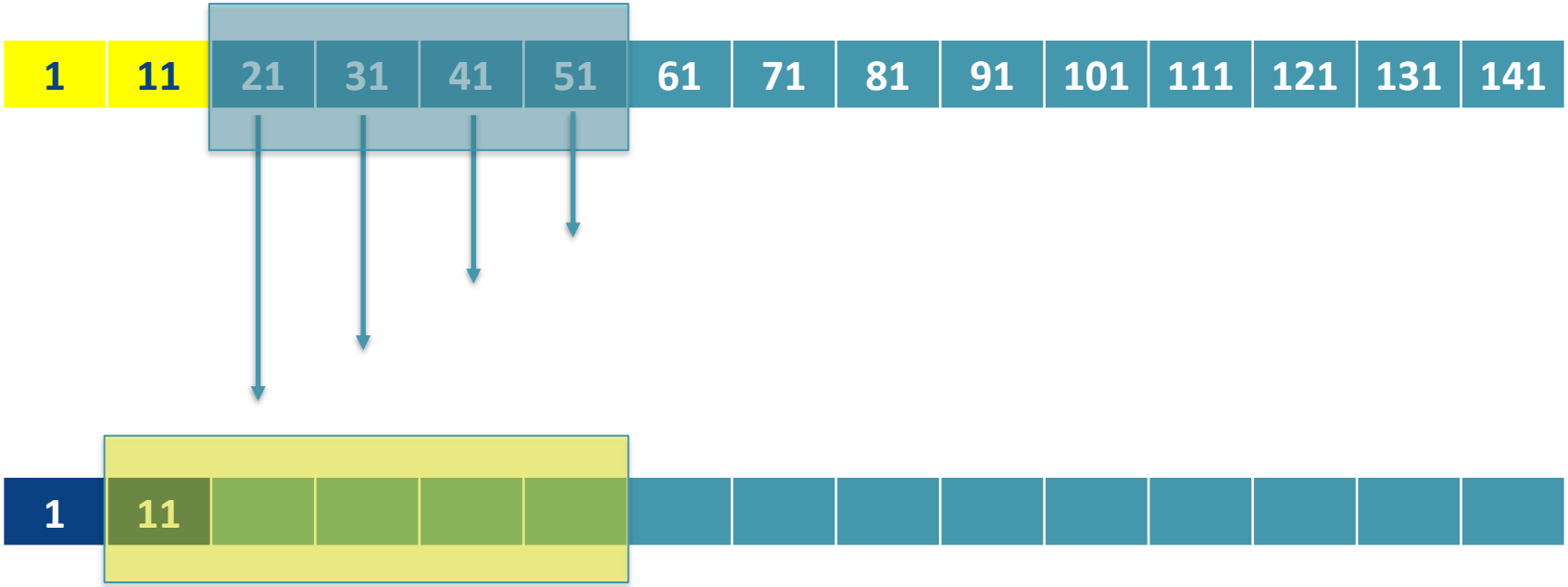
© 2025 University of Melbourne

TCP Sliding Window



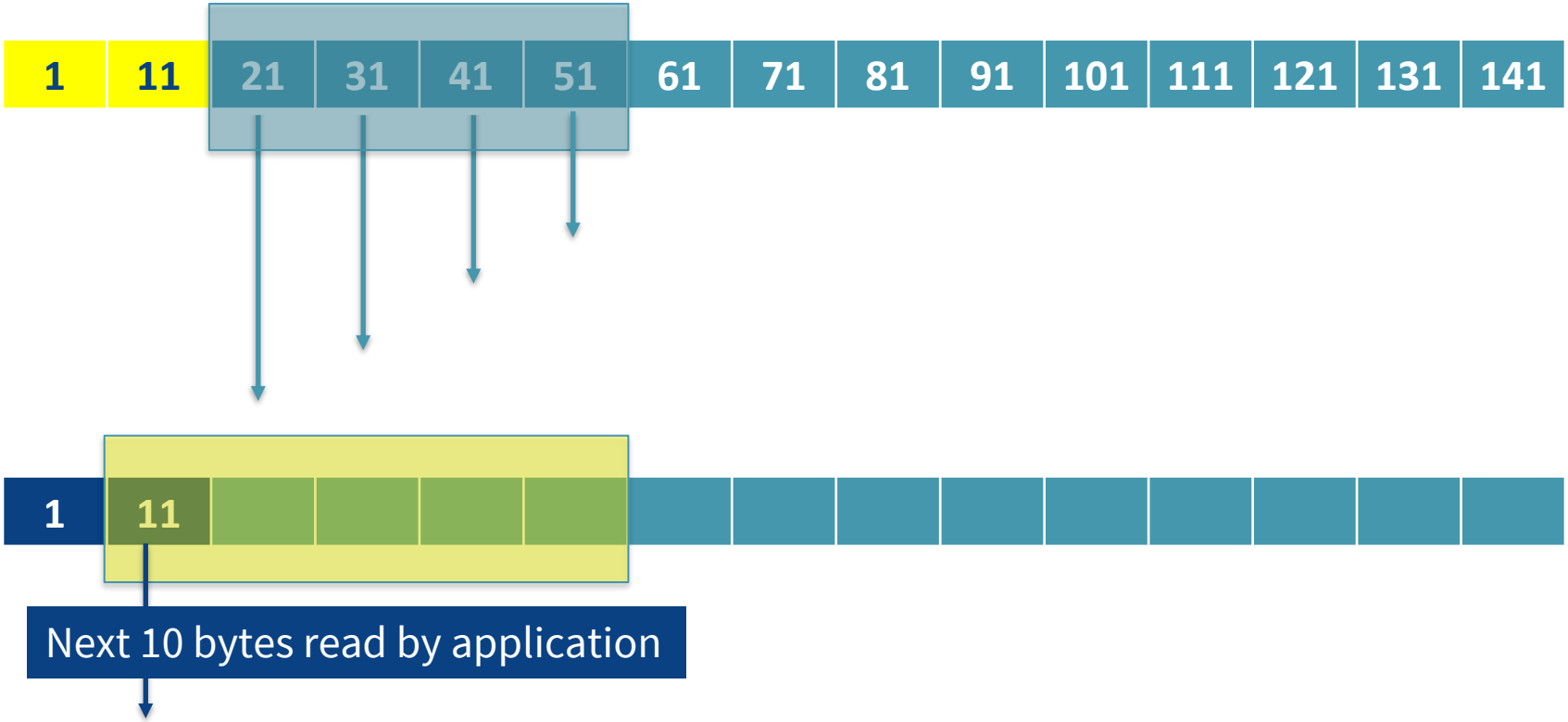
© 2025 University of Melbourne

TCP Sliding Window



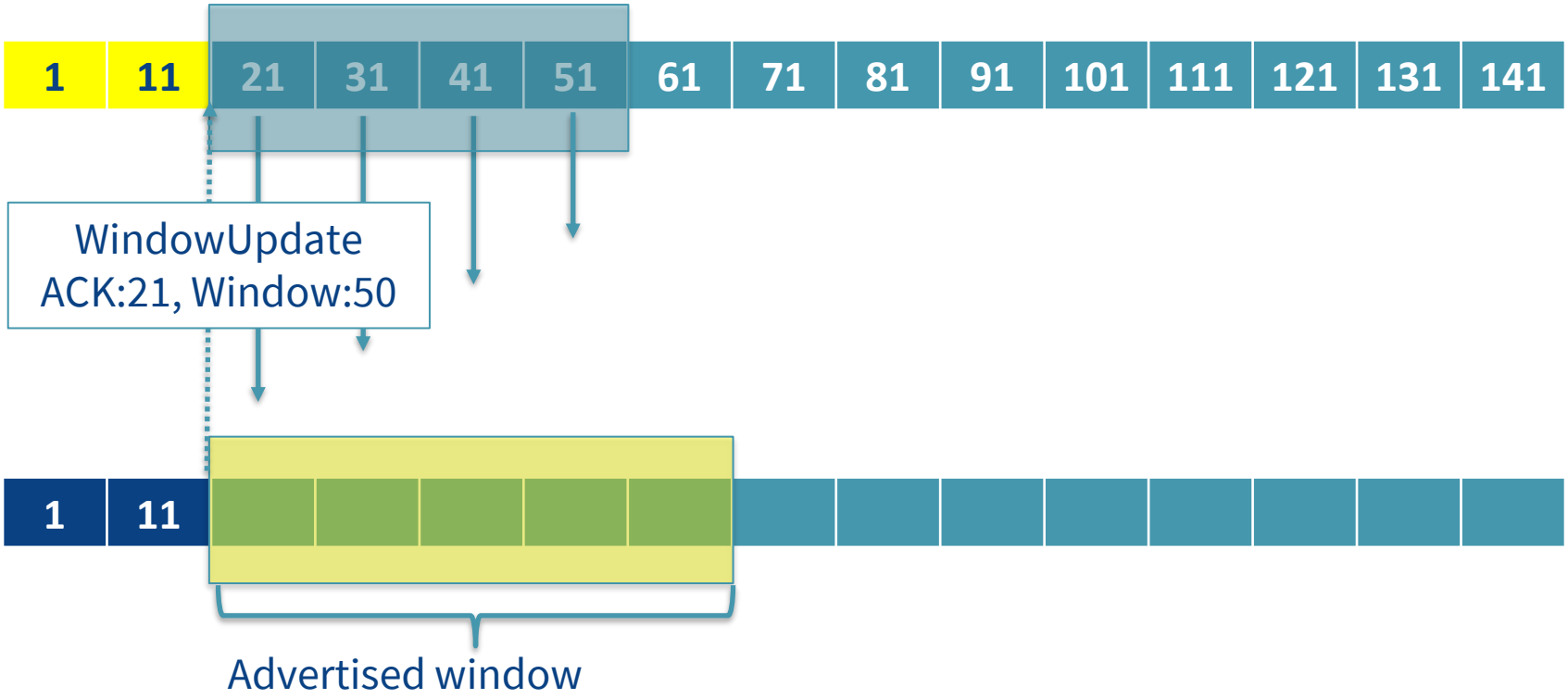
© 2025 University of Melbourne

TCP Sliding Window



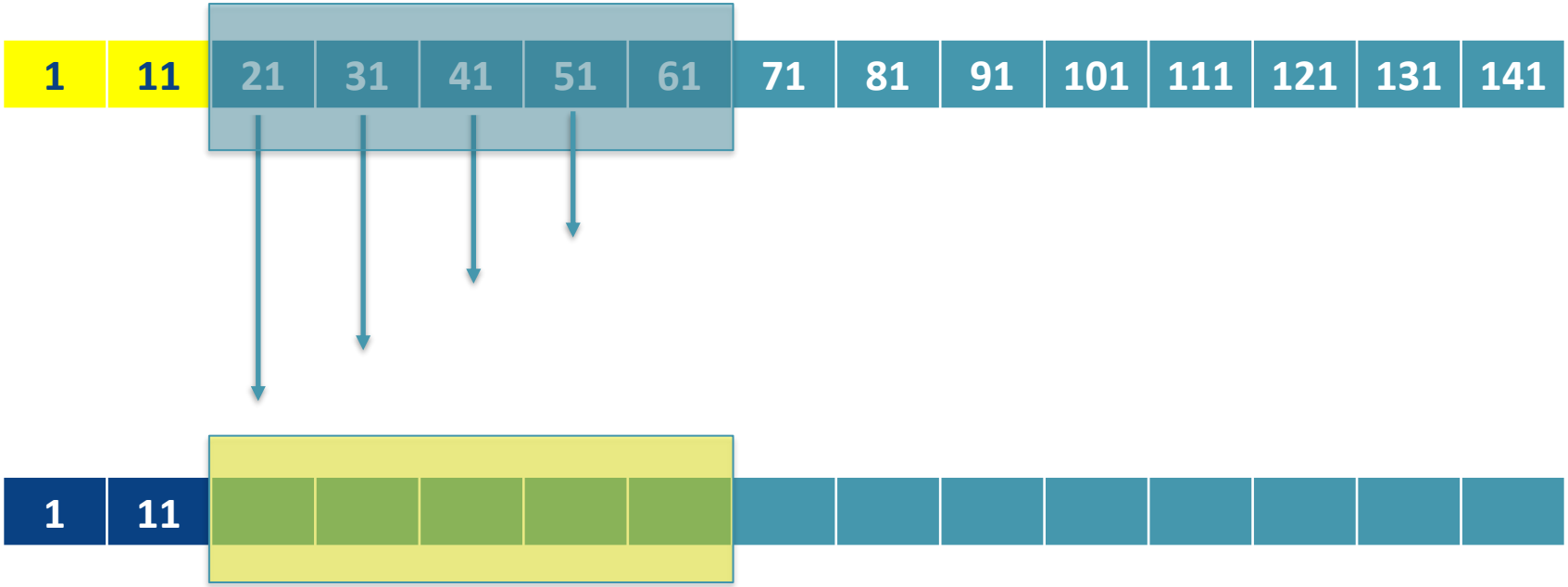
© 2025 University of Melbourne

TCP Sliding Window – Window Update



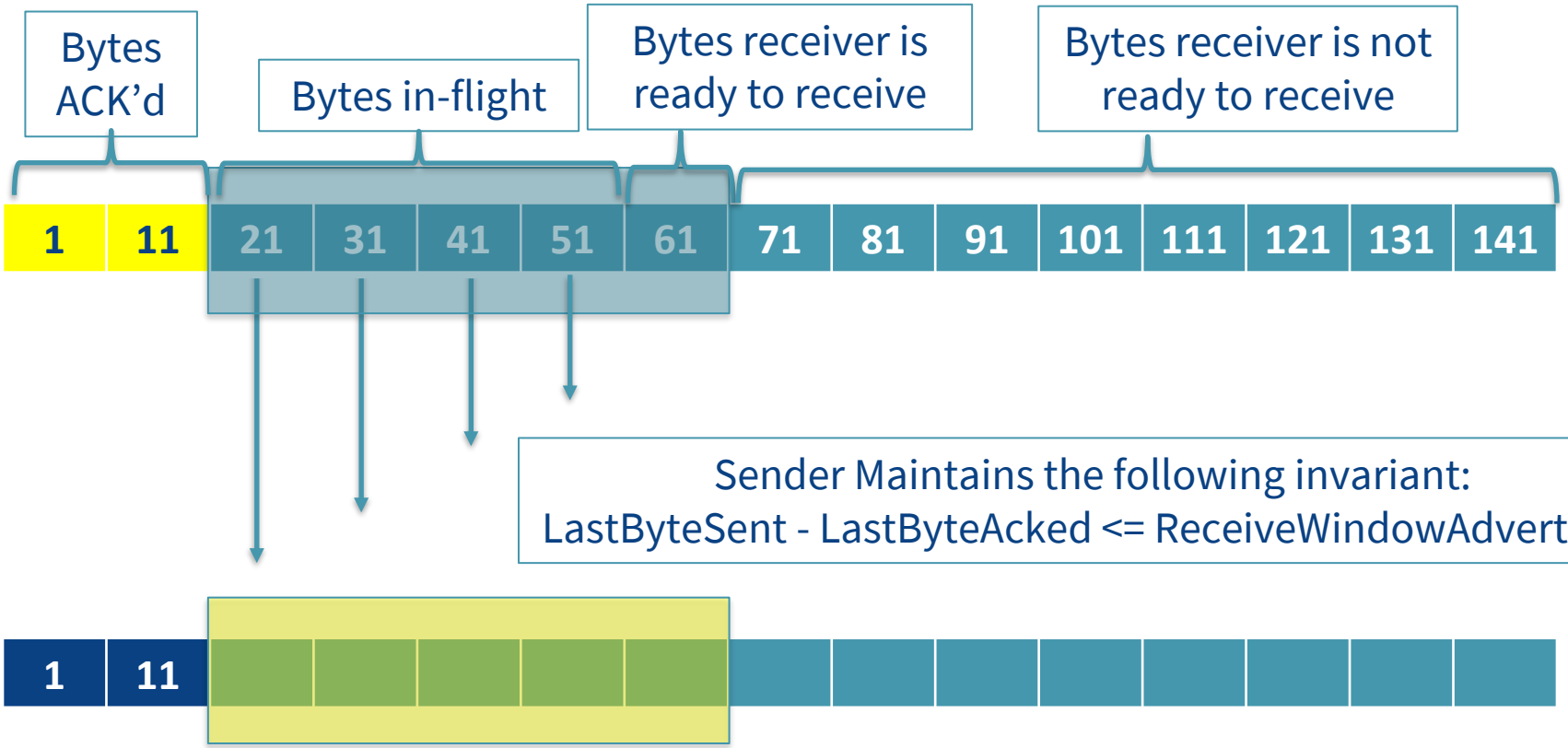
© 2025 University of Melbourne

TCP Sliding Window



© 2025 University of Melbourne

TCP Sliding Window

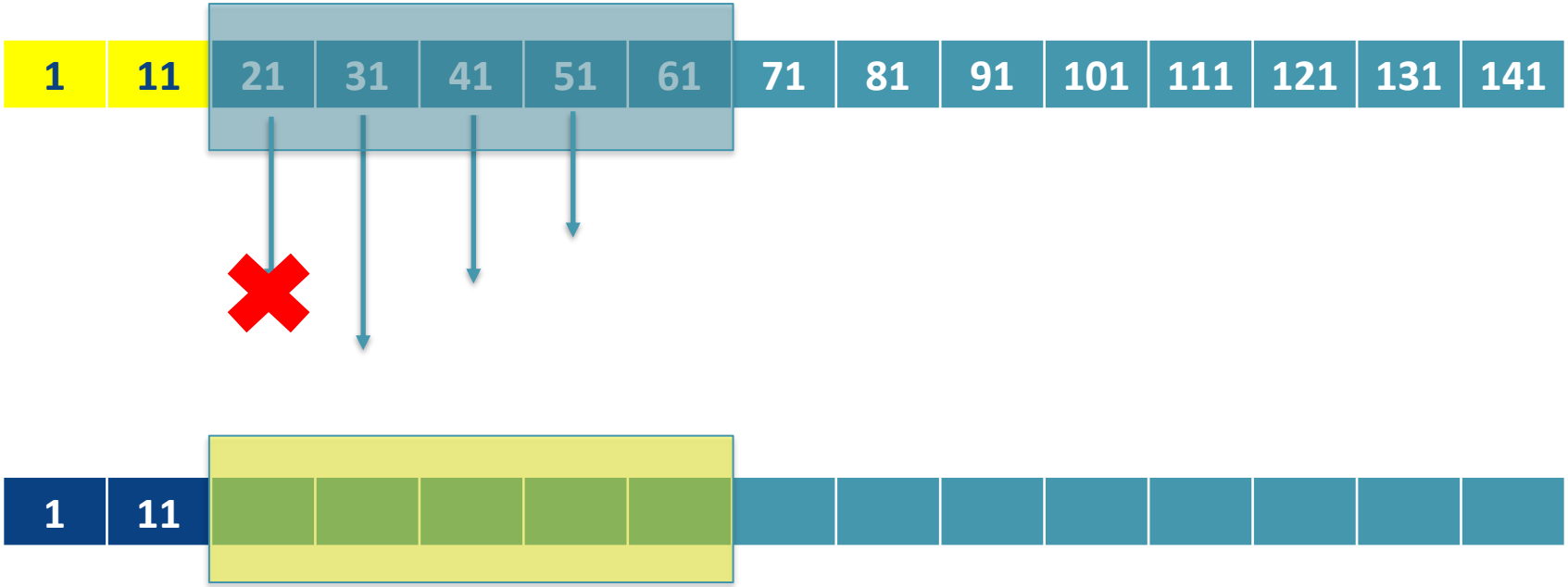


TCP Sliding Window

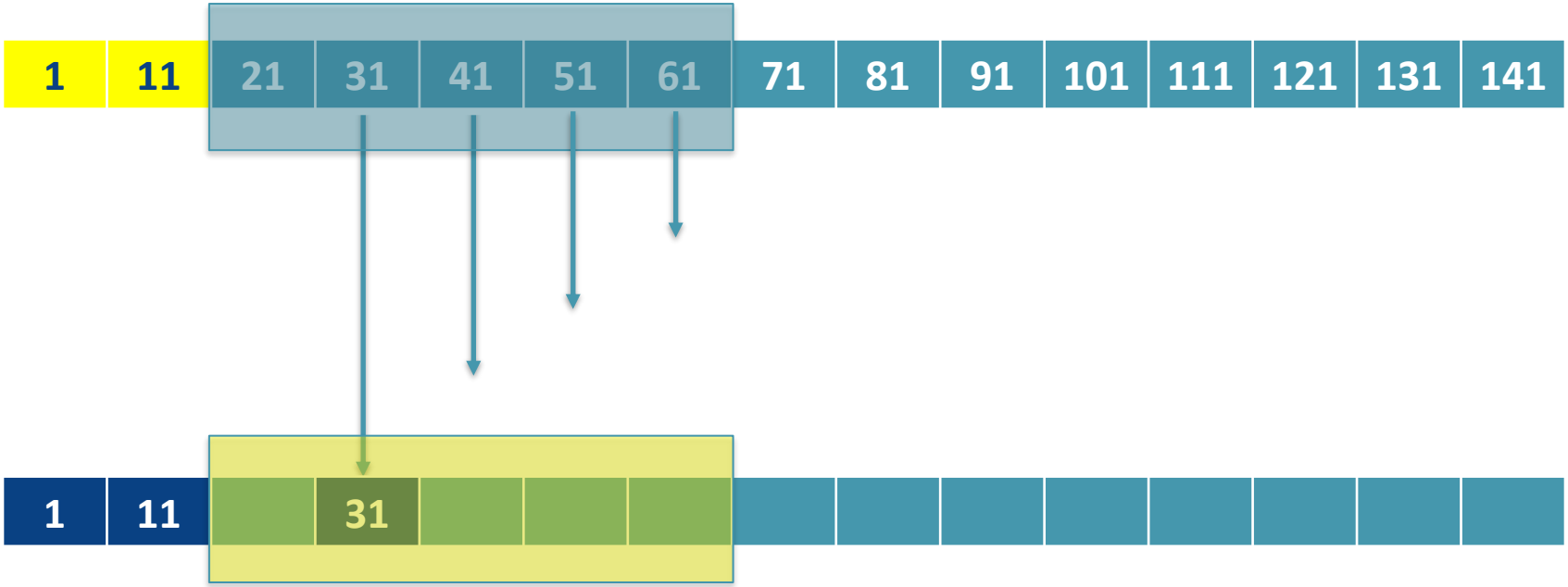
- Sliding window provides
  - Flow control (not sending too much to the receiver) ✓
  - Reliable delivery
  - In-order delivery

© 2025 University of Melbourne

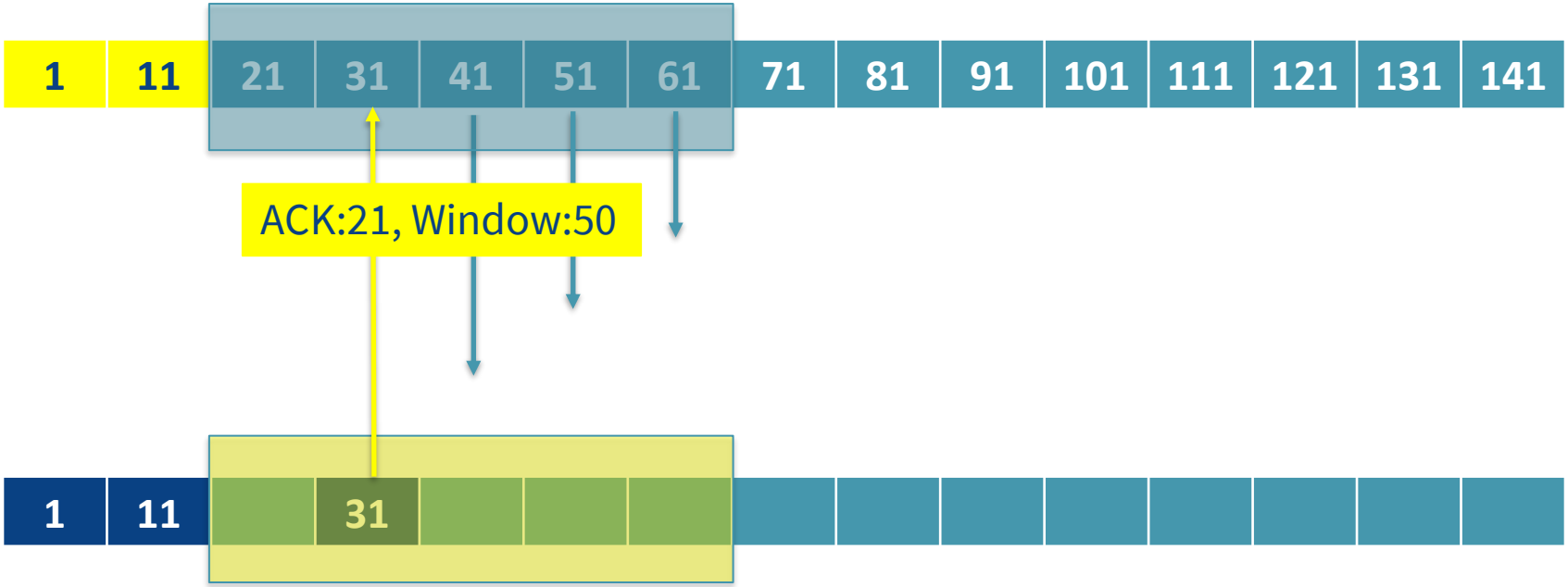
TCP Sliding Window: Segment Loss



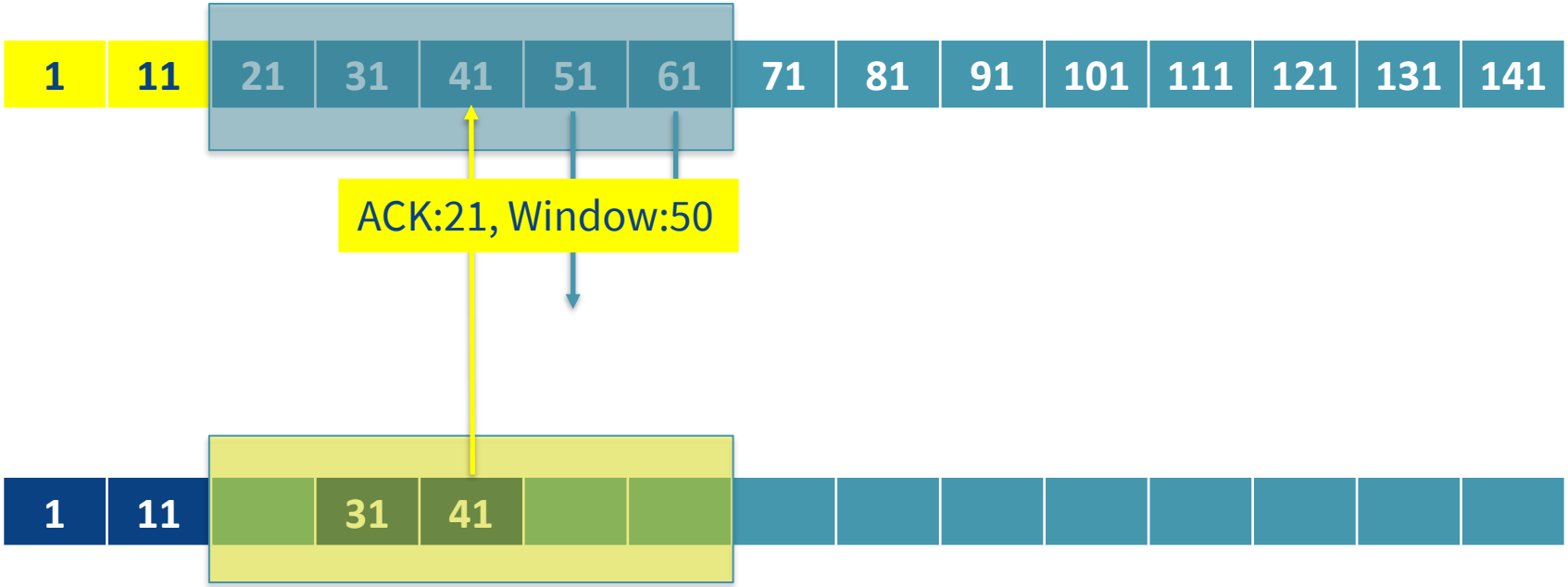
TCP Sliding Window: Segment Loss



TCP Sliding Window: Segment Loss

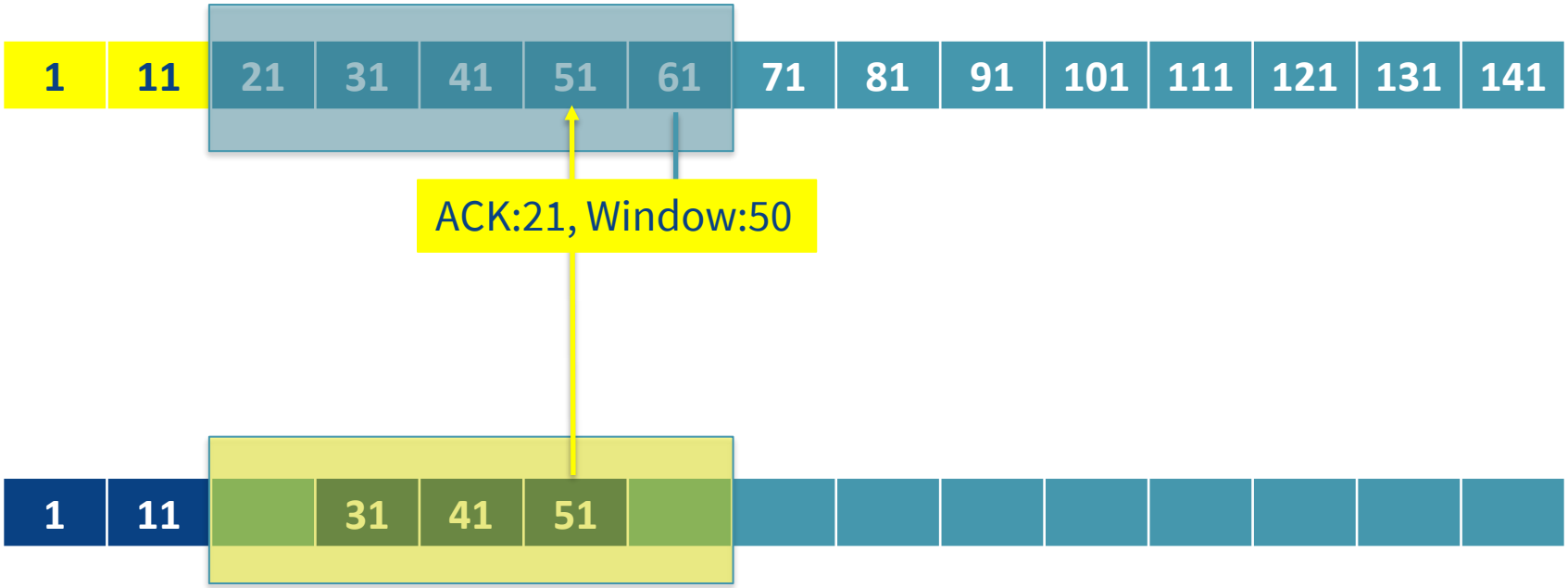


TCP Sliding Window: Segment Loss



TCP Sliding Window: Segment Loss

ACK + 3 DupACKs = Fast Retransmission



Original flow + loss control

- Flow+loss control had existed on single point-to-point links for a long time
- TCP originally used the experience from the link layer
- Caused bad design decision.

Two types of flow control

- "Go-back-N" (3.4.3 of Kurose and Ross)
  - When a packet is lost, go back to the point where it was transmitted, and (re)transmit everything from that point onward
  - Advantage: Receiver doesn't need to store/reorder packets
- "Selective Repeat" (3.4.4 of Kurose and Ross)
  - Only retransmit the lost packet
  - Packets arrive out of order. Receiver must store out-of-order packets to send them in-order to the application
- Selective repeat: more complex, only helps if loss is common
  - Link layer fixes errors. Errors will be rare – right?

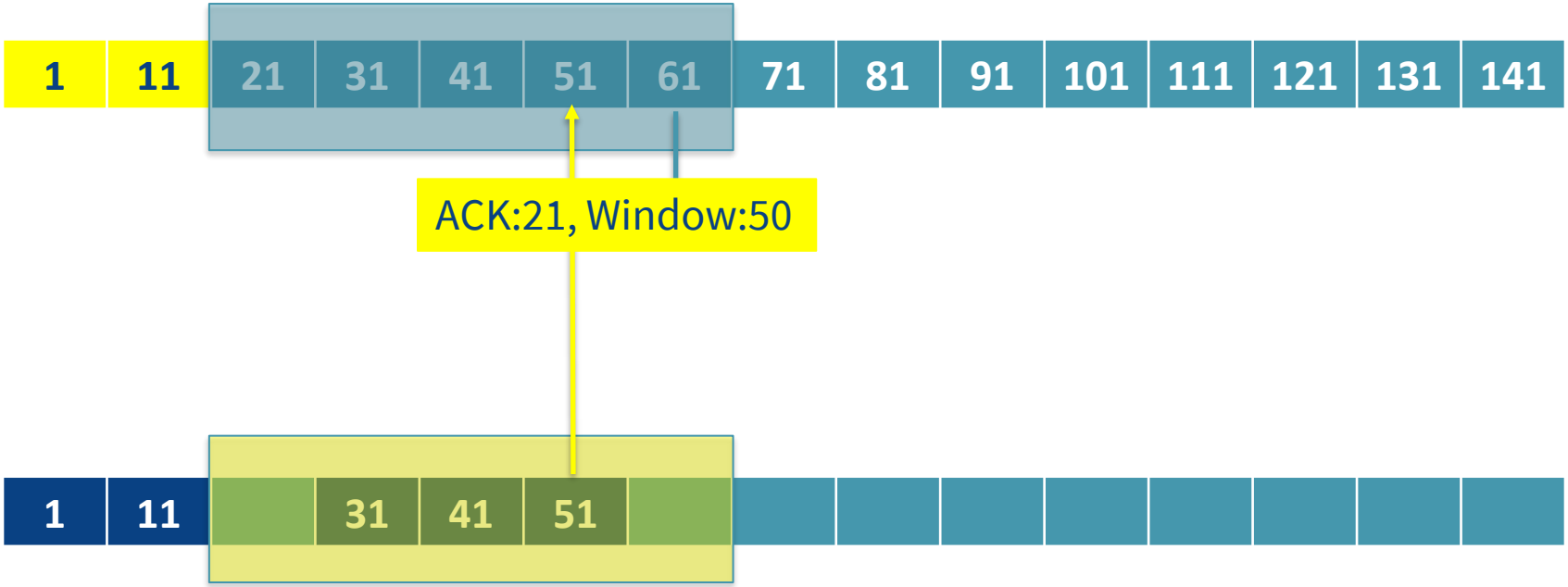
Congestion collapse

- In the late 1980s, the internet had “congestion collapse”
  - It took tens of minutes to send a packet to the next building
- Van Jacobson diagnosed and solved the problem

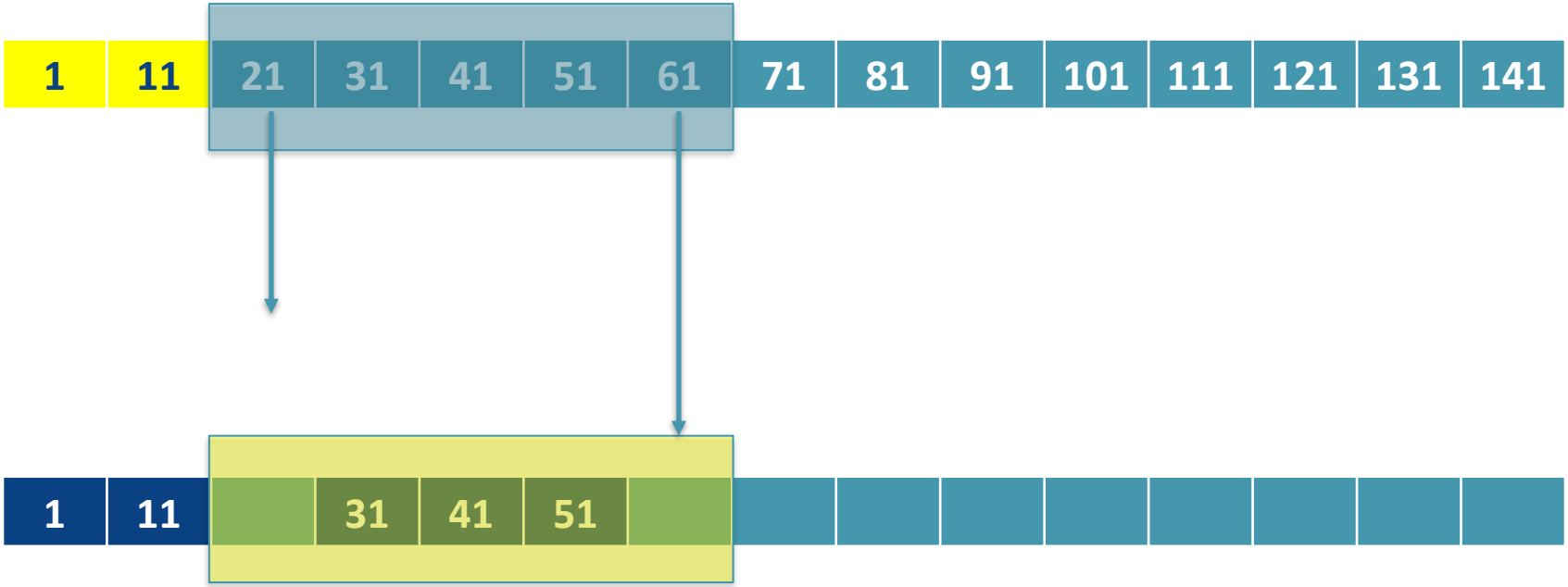
- Router buffers were overflowing, causing high loss
- Senders were doing go-back-N, so that every packet loss caused N more packets to enter the system
- Solution: Selective repeat (“fast retransmit”)
  - “Packet conservation” principle

TCP Sliding Window: Segment Loss

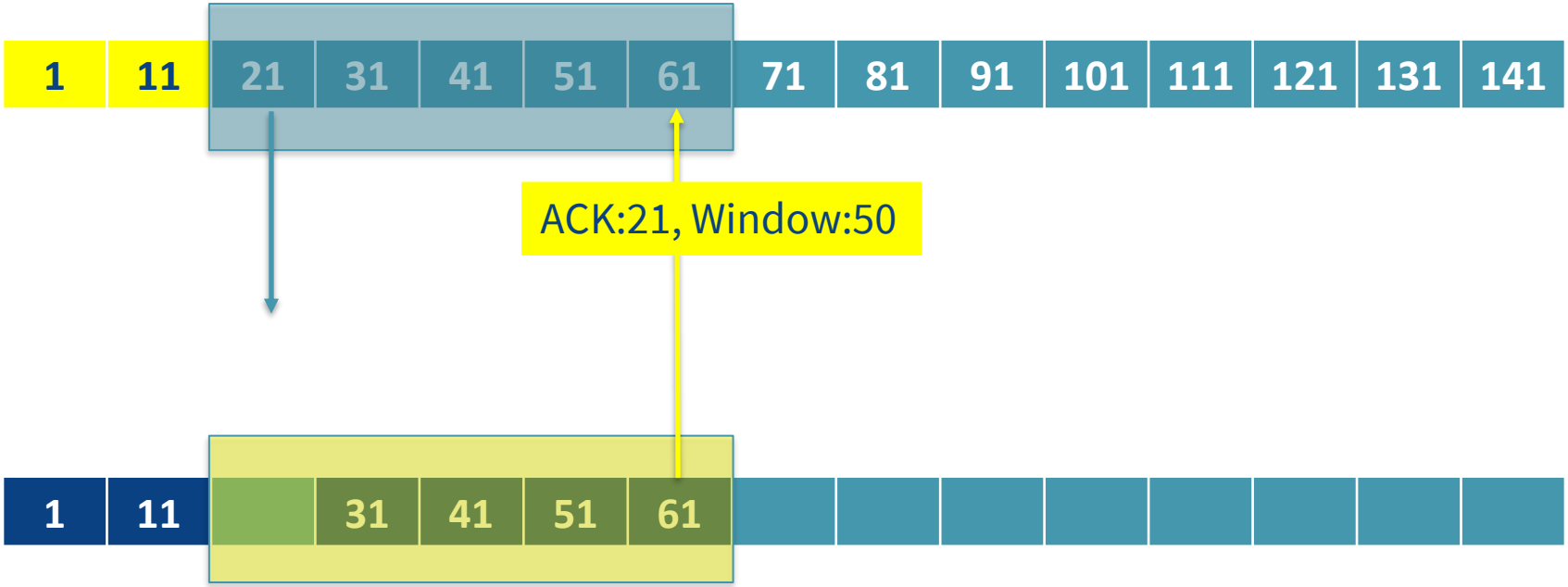
ACK + 3 DupACKs = Fast Retransmission



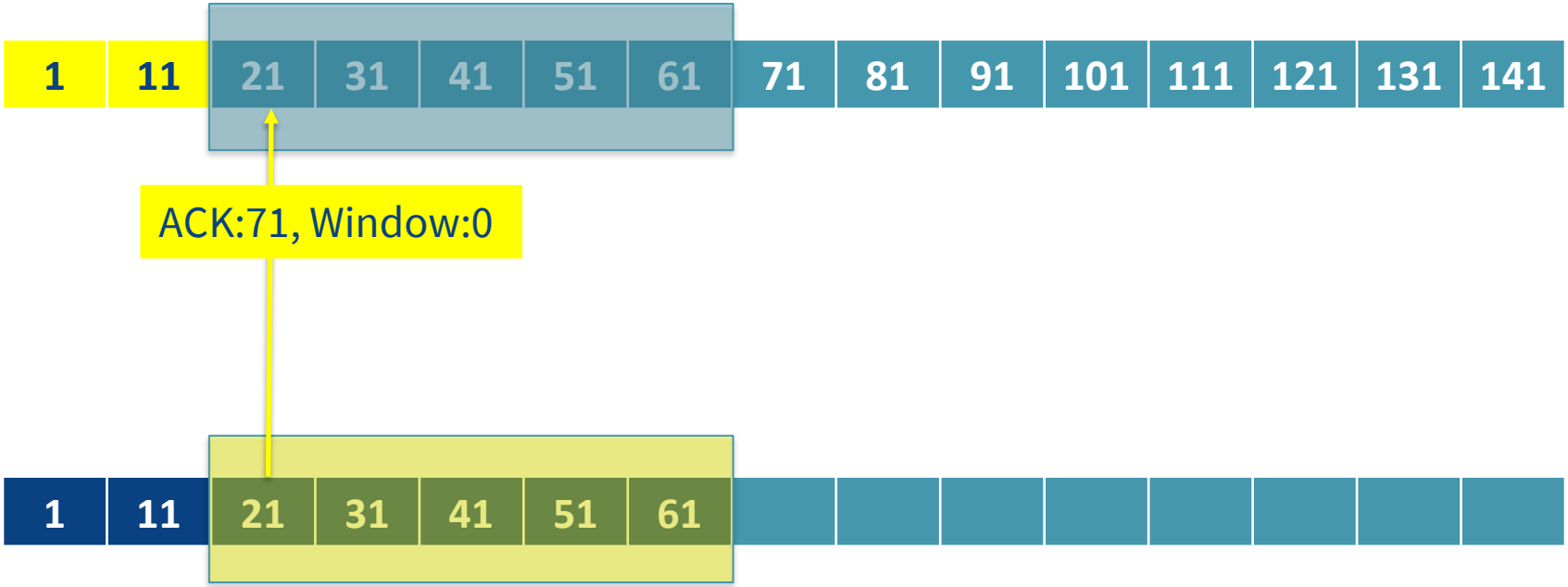
TCP Sliding Window: Fast retransmit



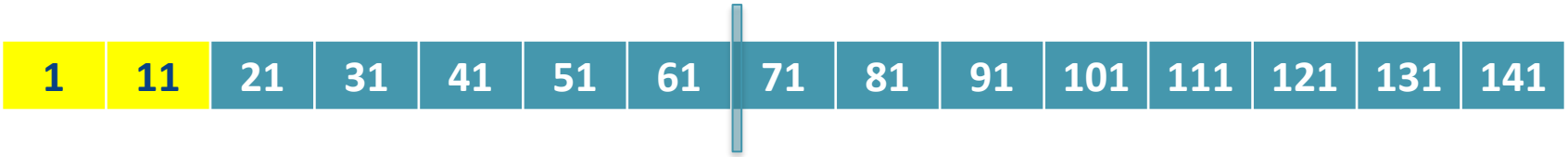
TCP Sliding Window: Fast retransmit



TCP Sliding Window: Fast retransmit



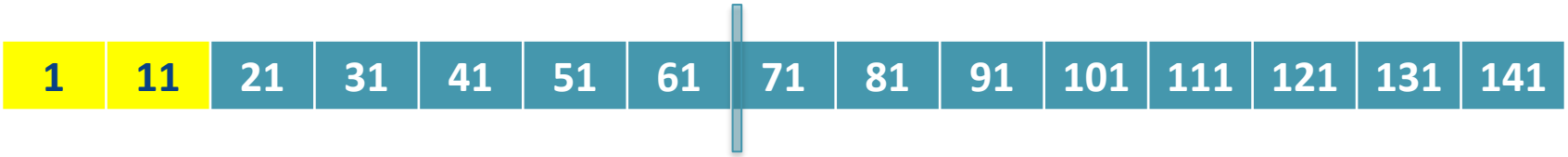
TCP Sliding Window

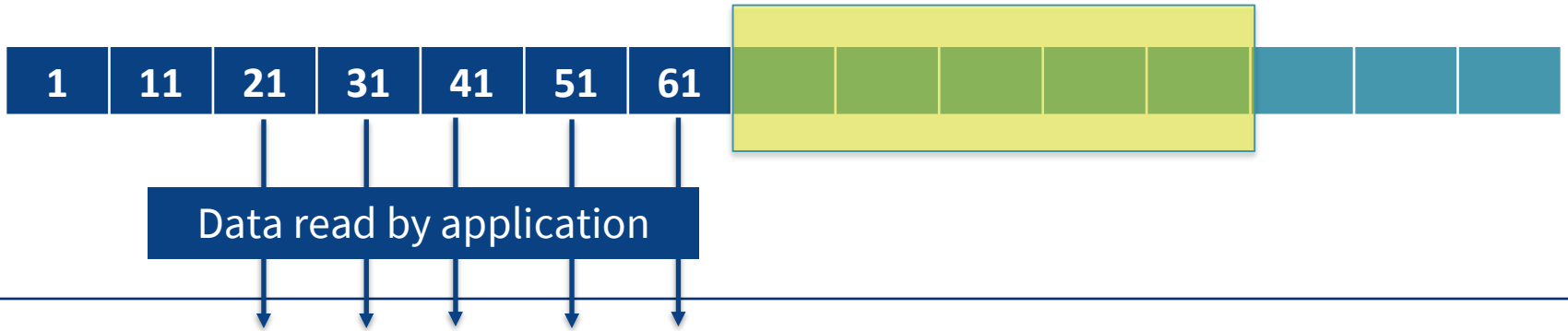


Potential for deadlock

- Sender won't send any more data (window size is 0)
- Receiver won't receive anything, so won't send any ACKs to increase window size

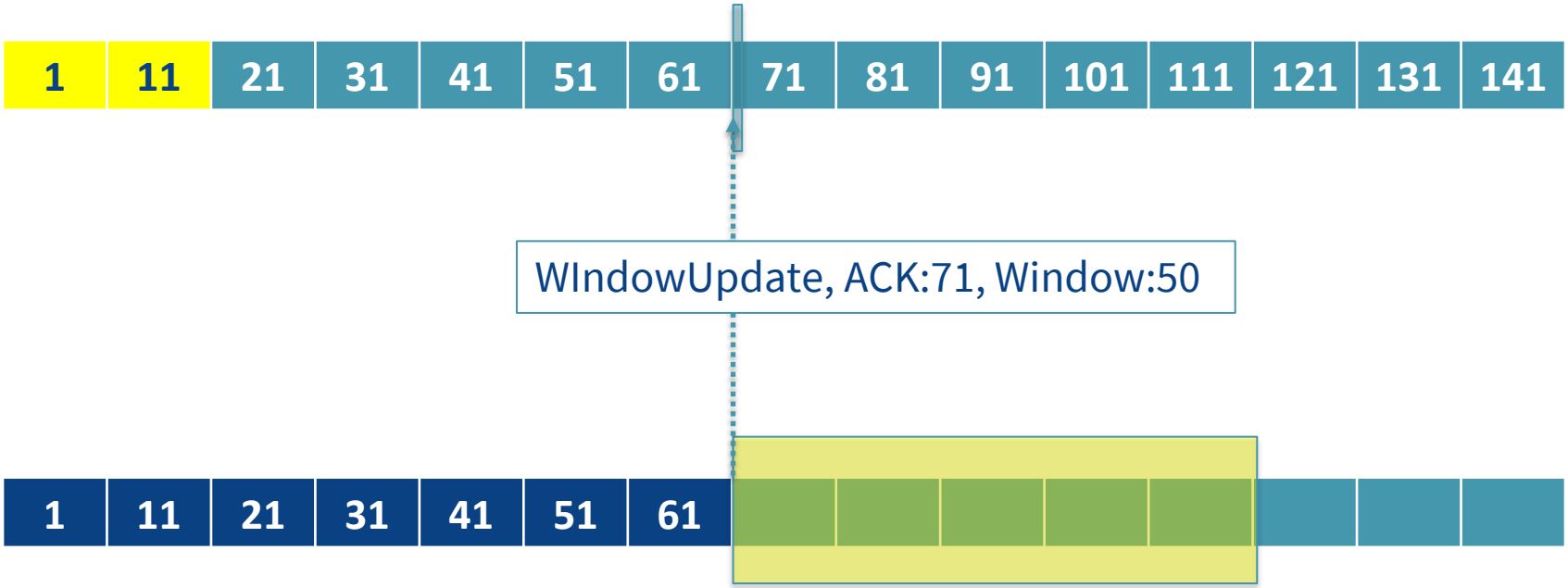
TCP Sliding Window: Avoid Deadlock



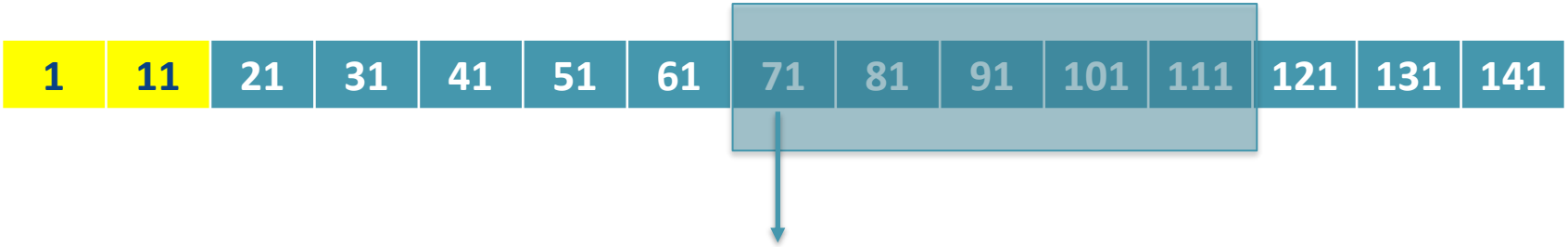


TCP Sliding Window: Window Update

Receiver can initiate an update by sending a WindowUpdate



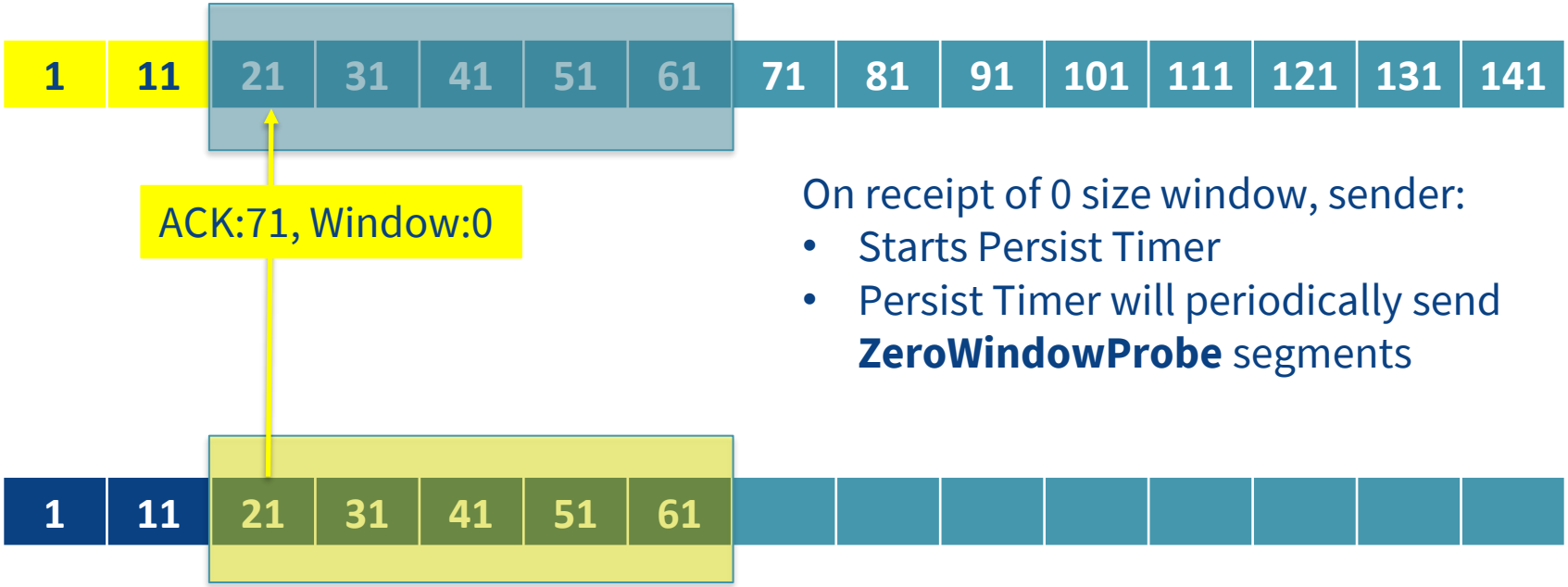
TCP Sliding Window: Window Update



© 2025 University of Melbourne

TCP Sliding Window: Persist Timer

Sender can monitor and initiate an update by sending a ZeroWindowProbe

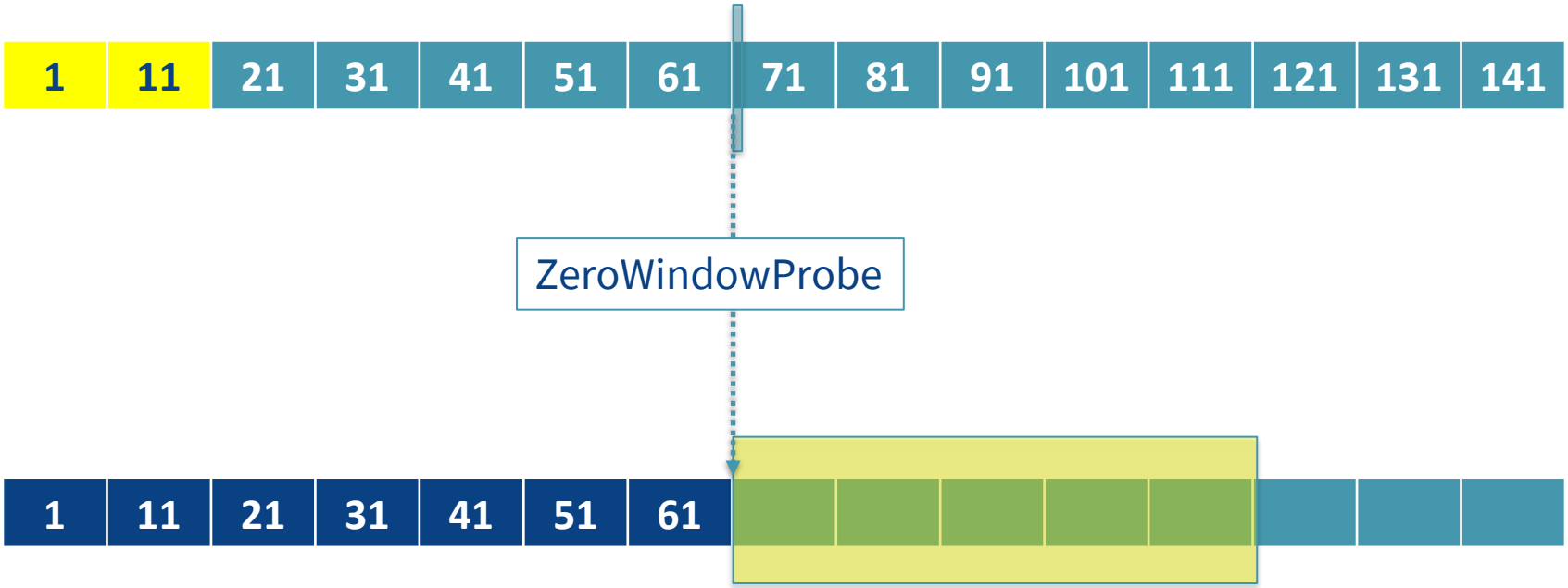


On receipt of 0 size window, sender:

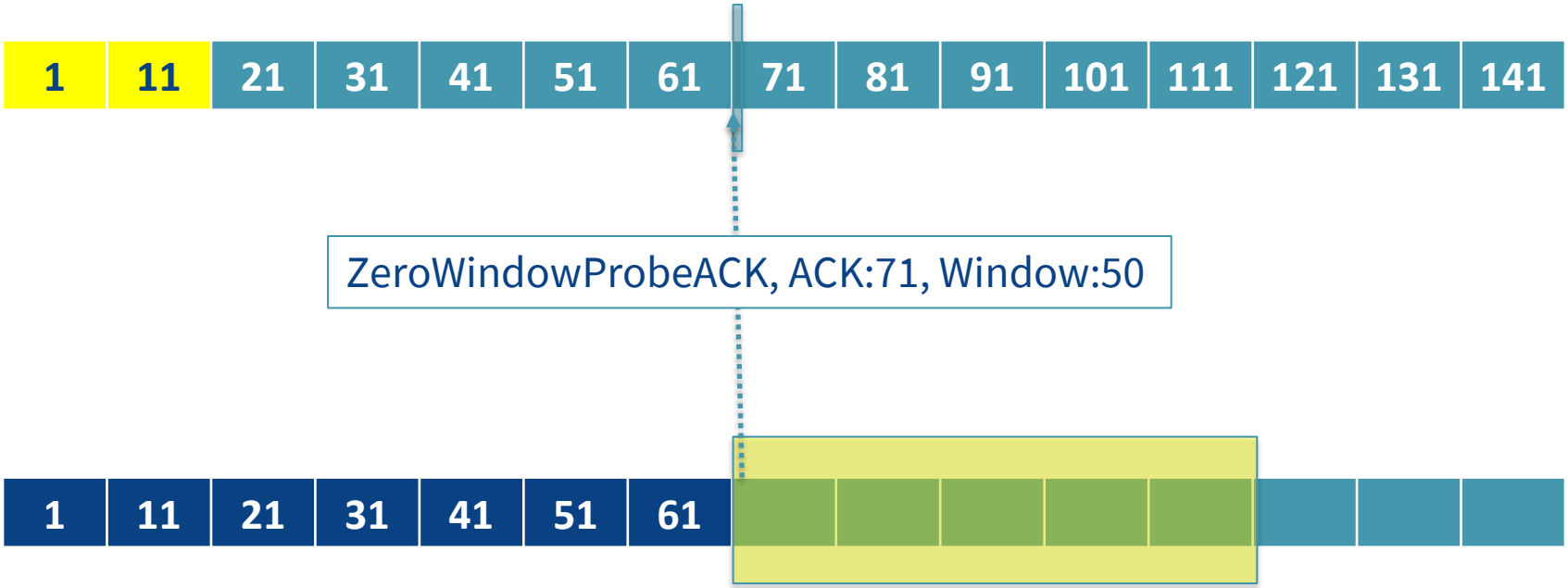
- Starts Persist Timer
- Persist Timer will periodically send **ZeroWindowProbe** segments

TCP Sliding Window

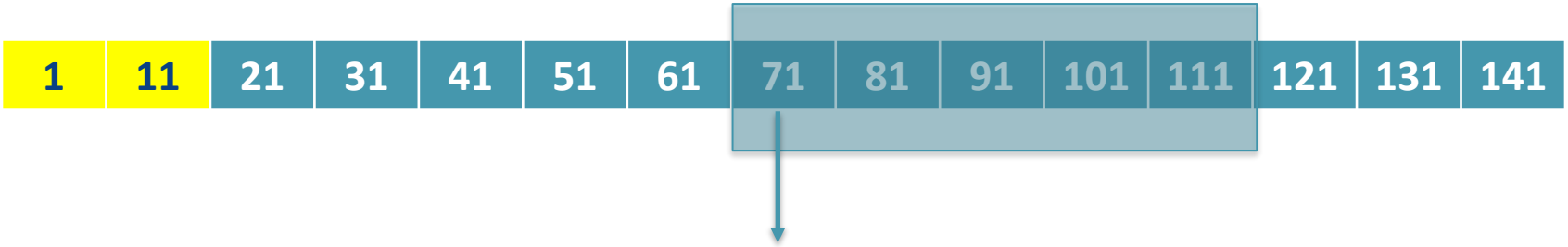
Sender can initiate an update by sending a ZeroWindowProbe



TCP Sliding Window



TCP Sliding Window



© 2025 University of Melbourne

TCP Congestion Control

- When networks are overloaded, congestion occurs, potentially affecting all layers
- Although lower layers (link and network) attempt to reduce congestion, in reality TCP affects congestion most significantly because TCP offers methods to transparently reduce the data rate, and hence reduce congestion itself
  - Real-time control protocol also helps by changing video compression
- Original TCP (before Jacobson)
  - Initially, the receiver chooses a window based on its buffer size
  - if the sender is constrained to this size, then congestion problems will not occur due to buffer overflow at the receiver itself, but may still occur due to congestion within the network

Congestion Control Window

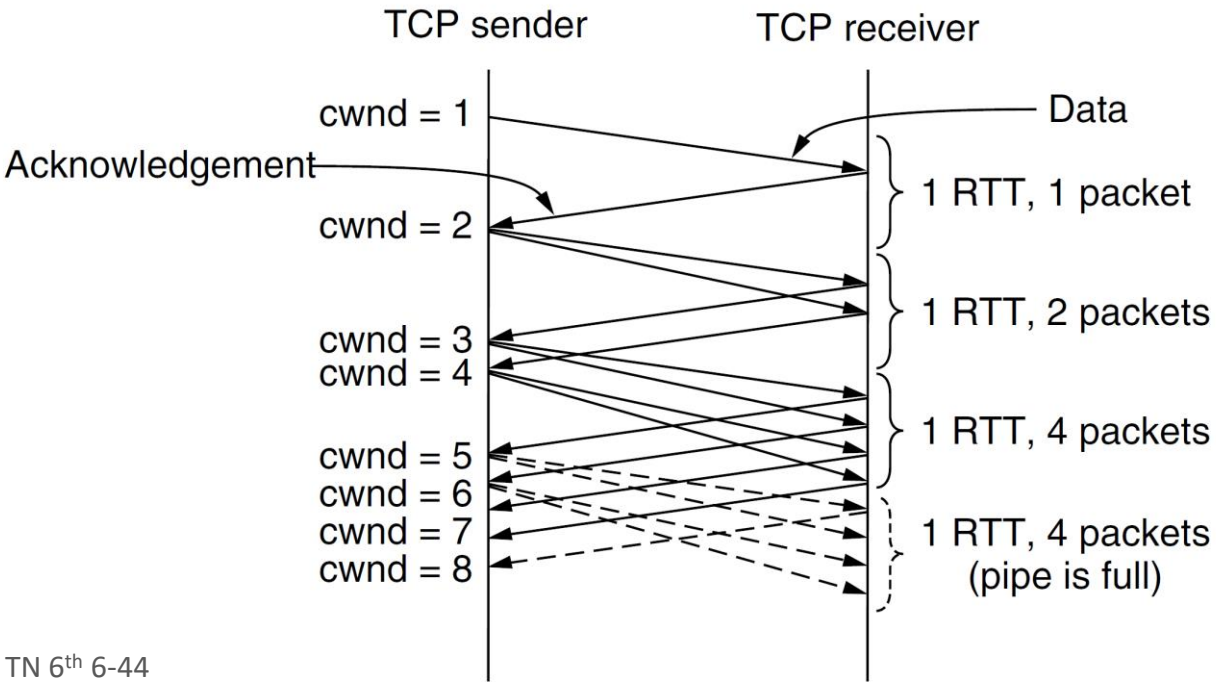
- Jacobson introduce CWND, the congestion window, in the same software update that introduce selective repeat.
- Additional window that is dynamically adjusted based on network performance to aid efficient transfer
- The congestion window size is maintained by the sender, unlike the sliding window that is controlled by the receiver
  - Also only used at the sender.
  - No changes to packet formats to send additional field
    - Was a big consideration. Now a huge one.

Incremental congestion control

- At first, CWND ← maximum segment size
  - Sender transmits 1 segment
- For each segment acknowledged
  - CWND += maximum segment size (MSS)
  - Sender transmits *2* more segments
- Each full window of acknowledgements *doubles* the congestion window - which grows until
  - a timeout
  - It reaches a threshold, SSthresh

Incremental congestion control

- Slow-start algorithm
  - Initial rate is slow
  - Rate grows exponentially from there

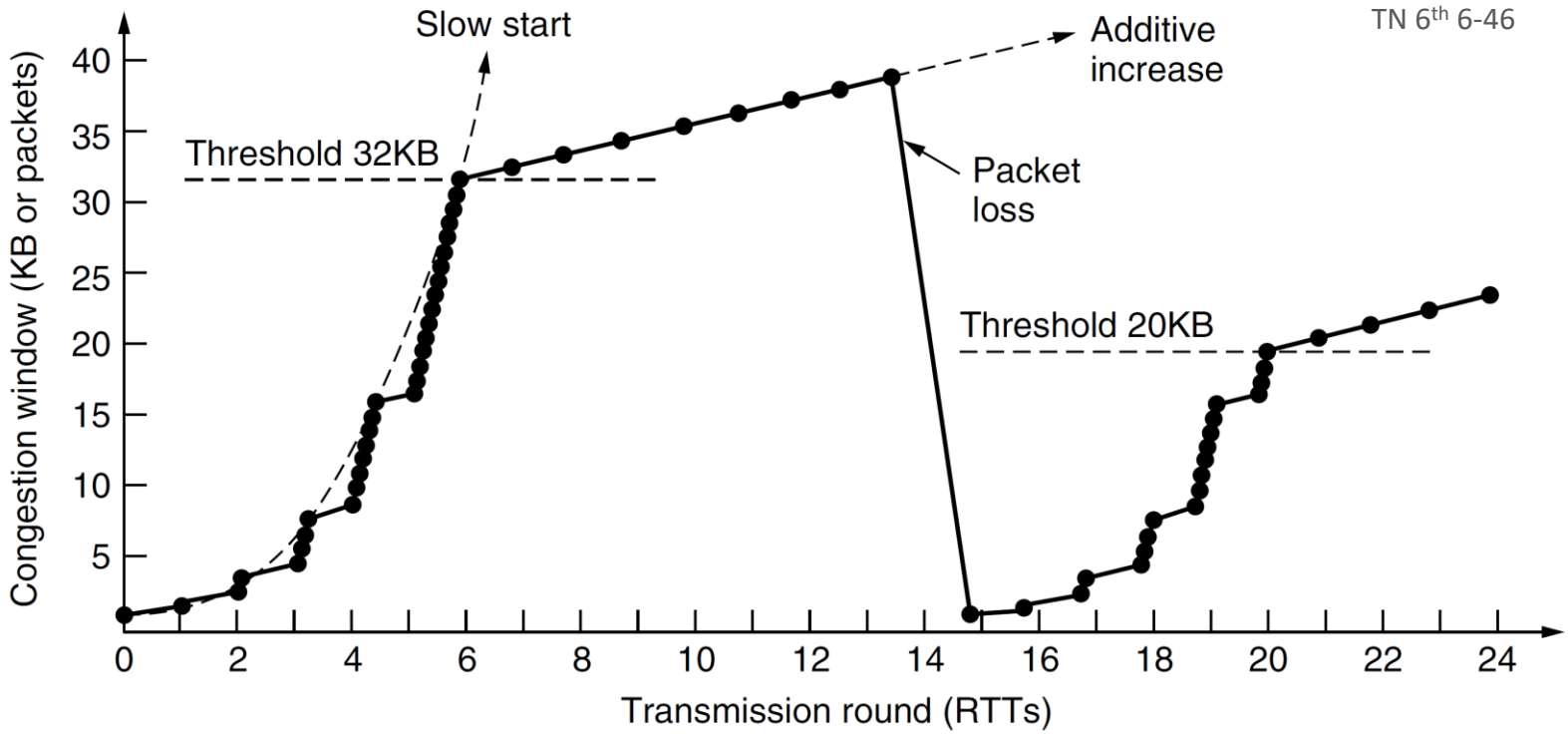


TN 6th 6-44

Incremental congestion control

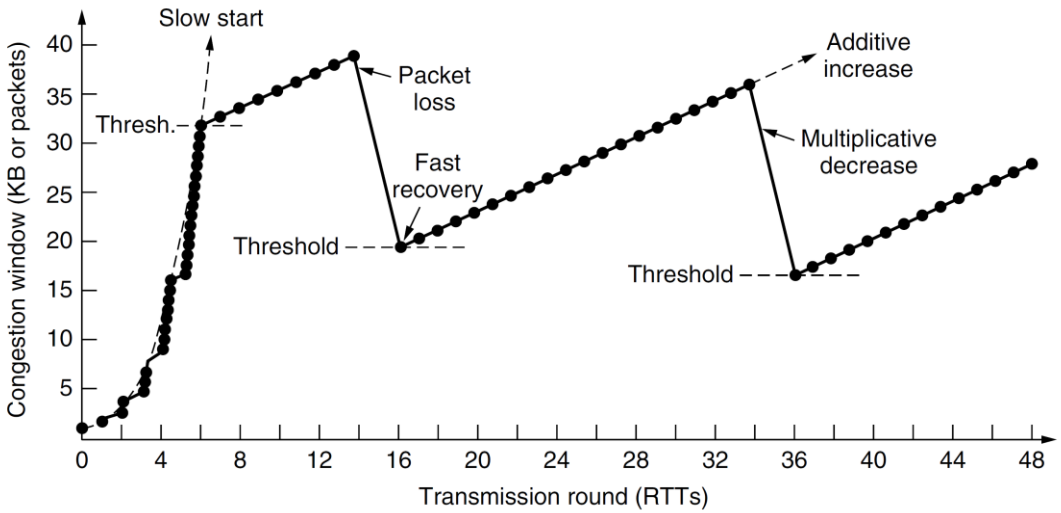
- Eventually too many segments would be placed onto the network causing congestion and timeouts
- Slow-start maintains a threshold: Slow Start Threshold ssthresh
- Slow start keeps increasing the size of the window until a timeout occurs or the threshold is reached
- On segment loss: SSthresh ← CWND/2, start slow start again
- Once the SSthresh is reached, the growth is slowed to linear
  - Achieved by CWND += MSS for each *complete window* of ACKs
  - "additive increase"
- Can also react to known lost segments via fast retransmission
- Known as TCP Tahoe (1988) after BSD release name

Incremental congestion control – an example



Congestion control optimisation

- Further optimisations were made in TCP Reno (BSD Reno)
- Fast recovery when fast retransmit is triggered
  - Starts from new ssthresh instead of original starting value, effectively avoiding the slow start phase and going straight to additive increase



TN 6th 6-47

And finally…

- For networks with very large windows (high speed, long distance), TCP Reno needs unrealistically small packet loss
  - New congestion control algorithms are need for bulk transfers between data centres on different continents
- Within data centres, needs are different again

- IETF won't change, in case things break
- Just as the internet pioneers bypassed the ISO, companies are implementing their own standards on their own networks
  - DCTCP for data centres, Google's BBR between data centres.

https://www.theregister.com/2021/10/27/systems_approach_tcp_control

© 2025 University of Melbourne

Acknowledgement

- The slides are based on slide prepared by Chris Culnane based on material developed previously by: Michael Kirley, Zoltan Somogyi, Rao Kotagiri, James Bailey and Chris Leckie.
- Some of the images included in the notes were supplied as part of the teaching resources accompanying the text books listed in lecture 1.
  - (And also) Computer Networks, 6th Edition, Tanenbaum A., Wetherall. D. https://ebookcentral.proquest.com/lib/unimelb/detail.action?docID=6481879
- Textbook Reference: 3.6, 3.7

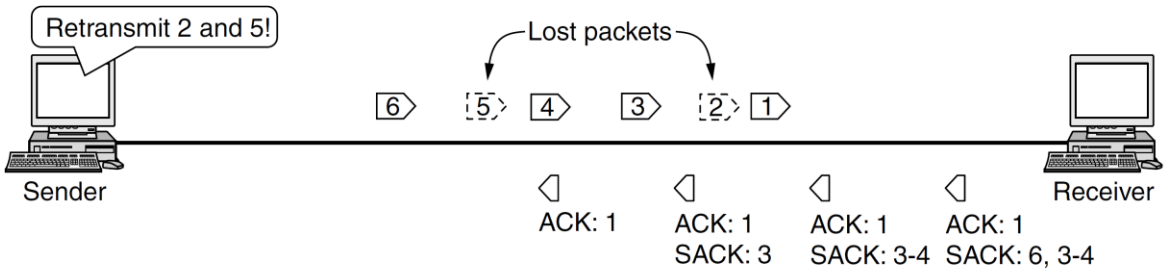
- The following slides are not examinable, but provide information you may find useful

© 2025 University of Melbourne

Further Optimisation

- SACK (Selective Acknowledgements) – provides greater ability to track segments in-flight, by allowing up to 3 ranges of bytes received to be specified

TN 6th 6-48



Macroscopic model

- These packet-level rules affect many things we care about
  - Fairness between flows
  - Response to long round-trip times (RTTs)
  - Response to random packet loss
- It is useful to have an algebraic expression relating these quantities

Window size

- W increases once per window
  - Approximation: Each packet that arrives increases W by 1/W
- When a loss occurs (with probability p), W is halved
  - With probability p, W ← W/2
- The average increase in window size is
- $\frac{(1-p)}{W} - p\frac{W}{2}$
- To be in equilibrium (balance), the average increase must be 0
  - $W \approx \sqrt{2/p}$

Rate, fairness

- The window is sent ≤ once per round trip time (RTT), T
- Rate is $\frac{W}{T} \approx \frac{1}{T}\sqrt{2/p}$

1. For a given packet loss rate, longer RTTs get less rate
   - "RTT unfairness"
2. If RTT is small, TCP forces the packet loss rate to be high

- This formula is very approximate
  - Window only responds to one packet loss per RTT
  - Packet losses are clustered
- However, it gives two important insights